

Dbms

Unit 3

1. What is the Semantics of Attributes?

Semantics of attributes refers to the **meaning and role of each attribute (column) in a relation (table)** in a database. It ensures that every attribute clearly represents a **single type of information about an entity**.

In the **Relational Database Model**, attributes describe the **properties or characteristics of an entity**.

Example

Consider a table **Student**:

Student_ID	Name	Age	Course
101	Rahul	20	BCA
102	Priya	21	BBA

Here:

- **Student_ID** → uniquely identifies a student
- **Name** → represents the student's name
- **Age** → student's age
- **Course** → program in which the student is enrolled

Each attribute has **clear semantics**, meaning it stores **one specific type of information**.

Important Rules of Attribute Semantics

1. Each attribute should represent **only one property** of an entity.
2. Attribute values should be **atomic (indivisible)**.
3. Attribute names should be **meaningful**.
4. Attributes should **not store redundant information**.

Bad Example (Poor Semantics)

Student_Info
Rahul, 20, BCA

Here one column stores **multiple values**, which makes interpretation difficult.

2. What is Normalization? Explain 1NF, 2NF, 3NF

Normalization is the process of **organizing data in a database to reduce redundancy and improve data integrity**.

It divides large tables into **smaller related tables** and establishes relationships between them.

Goals of Normalization

- Remove **data redundancy**
- Avoid **update anomalies**
- Improve **data consistency**
- Efficient database structure

Normalization is divided into **normal forms**.

First Normal Form (1NF)

A table is in **1NF** if:

1. All attributes contain **atomic (indivisible) values**
2. No **repeating groups**
3. Each field contains **single value**

Not in 1NF

Student_ID	Name	Phone
101	Rahul	98765, 91234

Phone contains **multiple values**.

In 1NF

Student_ID	Name	Phone
101	Rahul	98765
101	Rahul	91234

Each column contains **one value only**.

Second Normal Form (2NF)

A table is in 2NF if:

1. It is already in 1NF
2. There is **no partial dependency**

Partial dependency means a **non-key attribute depends on only part of a composite key**.

Example

Student_ID Course_ID Student_Name Course_Name

Primary Key = **Student_ID + Course_ID**

Problem:

- Student_Name depends only on **Student_ID**
- Course_Name depends only on **Course_ID**

This is **partial dependency**.

After 2NF

Student Table

Student_ID Student_Name

101 Rahul

Course Table

Course_ID Course_Name

C1 DBMS

Enrollment Table

| Student_ID | Course_ID |

Partial dependency removed.

Third Normal Form (3NF)

A table is in 3NF if:

1. It is already in 2NF

2. There is **no transitive dependency**

Transitive dependency means a non-key attribute depends on **another non-key attribute**.

Example

Student_ID Department_ID Department_Name

Here:

- Student_ID $\rightarrow$ Department_ID
- Department_ID $\rightarrow$ Department_Name

Thus:

Student_ID $\rightarrow$ Department_Name (indirect)

Solution (3NF)

Student Table

Student_ID Department_ID

Department Table

| Department_ID | Department_Name |

Transitive dependency removed.

3. What is Join? List and Explain Types of Join

A **Join** is an operation used to **combine rows from two or more tables based on a related column**.

Joins help retrieve **related information stored in multiple tables**.

Types of Join

1. Inner Join

Returns **matching rows from both tables**.

Example

Student Table

ID Name

- 1 Rahul
- 2 Priya

Marks Table

ID Marks

- 1 90
- 2 85

Query concept:

```
Student INNER JOIN Marks
```

Result:

ID Name Marks

- 1 Rahul 90
 - 2 Priya 85
-

2. Left Join (Left Outer Join)

Returns **all rows from left table** and matching rows from right table.

Example

ID Name Marks

- 1 Rahul 90
- 2 Priya NULL

Priya has **no marks record**.

3. Right Join (Right Outer Join)

Returns **all rows from right table** and matching rows from left.

Example

ID Name Marks

ID Name Marks

1 Rahul 90
3 NULL 80

4. Full Join

Returns **all rows from both tables**.

ID Name Marks

1 Rahul 90
2 Priya NULL
3 NULL 80

5. Cross Join

Returns **Cartesian product** of both tables.

Example

Table A:

A

1
2

Table B:

B

X
Y

Result

A B

1 X
1 Y
2 X
2 Y

4. Properties of Relational Decomposition

Relational Decomposition means **breaking a table into smaller tables** to eliminate redundancy.

Properties of Good Decomposition

1. Lossless Join Property

When decomposed tables are **joined back**, they must produce the **original table without loss of information**.

Example

Original Table

| Student_ID | Course_ID |

Decomposed into

- Student(Student_ID)
- Enrollment(Student_ID, Course_ID)

When joined → **original table restored**.

2. Dependency Preservation

All **functional dependencies** should still be maintained after decomposition.

Example

Original dependency

Student_ID → Student_Name

After decomposition this dependency should **still exist in some table**.

5. What is Lossless Join? Explain with Example

A **Lossless Join** means that when a relation is decomposed into multiple tables, the **original relation can be reconstructed by joining them without losing information**.

Example

Original Table

Student_ID	Student_Name	Department
101	Rahul	IT
102	Priya	CS

Decompose into two tables.

Table 1: Student

Student_ID	Student_Name
101	Rahul
102	Priya

Table 2: Department

Student_ID	Department
101	IT
102	CS

When we join:

`Student JOIN Department`

Result:

Student_ID	Student_Name	Department
101	Rahul	IT
102	Priya	CS

The **original table is perfectly reconstructed**, so it is a **Lossless Join**.

Lossy Join Example

If tables are decomposed incorrectly:

Table 1

| Student_ID | Student_Name |

Table 2

| Department |

There is **no common attribute**, so join will produce **incorrect data combinations**.

This is called **Lossy Join**.